CS6886 Assignment 5: Optimising attention on a CPU

Ritwik Mishra (ME23B237)

November 6, 2026

All times are the median of 5 runs after one warm-up, as printed by the unchanged driver. GFLOP/s uses the driver's count of 4N^2 d FLOPs, which covers the two matrix products and leaves out the exponentials and the max/sum. I ran everything on an Intel Core i5-12450H laptop under Ubuntu 24.04.5 LTS in WSL2 on Windows, compiled with Ubuntu clang 18.1.3 and `-O3 -march=native`. The laptop was plugged in and I closed other programs before the runs.

Q1. The machine

Field	Value
CPU model	Intel Core i5-12450H (12th Gen, laptop)
Microarchitecture	Alder Lake-H: P-cores Golden Cove, E-cores Gracemont
Physical cores	8 (4 P + 4 E)
Logical cores	12 (P-cores 2 threads each, E-cores 1 thread each)
Performance / efficiency cores	4 P (8 threads) + 4 E (4 threads)
Base clock	2.0 GHz P-core, 1.5 GHz E-core
Max single-core clock	4.4 GHz P-core (E-core max turbo 3.3 GHz)
All-core clock under load (measured)	2.81 GHz P-core, 2.54 GHz E-core
SIMD	AVX2, 256-bit (8 FP32 lanes), no AVX-512
FMA units per core	P-core 2 x 256-bit (32 FLOP/cycle); E-core 2 x 128-bit (16 FLOP/cycle)
Vector registers	16 ymm (256-bit) per hardware thread
L1 data cache	48 KB per P-core, 32 KB per E-core
L2 cache	1.25 MB per P-core; 2 MB shared by the 4 E-cores
L3 cache	12 MB, shared by all 8 cores
Cache line	64 B
RAM	16 GB DDR4-3200, 2 x SODIMM, 2 channels
OS	Ubuntu 24.04.5 LTS on WSL2 (kernel 6.6.87.2-microsoft-standard-WSL2)
Compiler	Ubuntu clang 18.1.3

WSL2 doesn't show the real topology. Inside it, `lscpu` reports 6 cores with 2 threads each, a uniform 48 KB L1d and 1.25 MB L2 per core, and only 7.6 GiB of RAM, which is the share Windows gives the VM. So I took the core counts, clocks and cache sizes from Intel ARK for the i5-12450H, and the RAM details from Windows Task Manager.

ARK doesn't list all-core clocks, so I measured them. I started one thread per logical CPU, 12 in all, each running a chain of dependent register-register integer adds. An add takes one cycle, so adds per nanosecond gives the clock in GHz. Logical CPUs 6 to 9 were always slower than the rest, so I took them to be the four E-cores and the other eight to be the P-core threads. The medians over three runs were 2.81 GHz for P and 2.54 GHz for E. My first try used add-immediate instructions and reported 18 GHz. Golden Cove folds immediate adds before they execute, so I dropped that version.

Peak FP32 throughput

A P-core has 2 FMA units. Each one works on a 256-bit AVX2 register, which holds 8 floats, and an FMA counts as 2 FLOPs. At the 4.4 GHz single-core turbo:

peak_1 = 4.4 GHz x 2 FMA/cycle x 8 lanes x 2 FLOP/FMA = 140.8 GFLOP/s

For all cores I used each core type's measured all-core clock. An E-core splits a 256-bit FMA into two 128-bit halves, so it gets 16 FLOP/cycle instead of 32. Hyperthreads share their core's FMA units and add nothing to the peak.

peak_all = 4 x 32 x 2.81 + 4 x 16 x 2.54 = 359.7 + 162.6 = 522.2 GFLOP/s

With every core at its ARK turbo clock this would be 4 x 32 x 4.4 + 4 x 16 x 3.3 = 774.4 GFLOP/s, but the chip can't hold those clocks on all cores at once.

I never saw 4.4 GHz in practice. On a single thread, the add chain and a chain of dependent `vfmadd231ss` (4-cycle latency) both gave about 3.0 GHz on every logical CPU. My guess is that the Windows power settings or the WSL2 hypervisor keep it there. At 3.0 GHz the single-core peak is 3.0 x 32 = 96 GFLOP/s. The best single-threaded variant gets 67.3 GFLOP/s at N = 4096, d = 64, which is 47.8% of the 140.8 GFLOP/s spec peak and 70% of the 96 GFLOP/s measured-clock peak. Over the four inputs, best is at 41.5% to 50.0% of the spec peak, or 61% to 73% of the measured-clock one. The tables below use 140.8 GFLOP/s.

Memory bandwidth and ridge point

The RAM is DDR4 at 3200 MT/s on 2 64-bit channels, and each channel moves 8 bytes per transfer:

BW = 3200 x 10^6 x 8 B x 2 = 51.2 GB/s

This is the theoretical figure. A STREAM-style copy usually gets 70-85% of it.

The ridge point is the intensity above which a kernel can be compute-bound. It is 140.8 / 51.2 = 2.75 FLOP/B for one core and 522.2 / 51.2 = 10.2 FLOP/B for all cores. One core needs very little reuse to stop being bandwidth-bound, while using every core needs 3.7x more.

Q2. Naive baseline

N	d	time (ms)	GFLOP/s	% of 1-core peak	AI (FLOP/B)
1024	64	126.21	2.13	1.5%	7.8
2048	64	534.93	2.01	1.4%	7.9
4096	64	2165.96	1.98	1.4%	7.9
4096	128	7827.40	1.10	0.8%	15.8

Naive gets 0.8% to 1.5% of one core's peak. Time grows as N^2: doubling N multiplies it by 4.2 and then 4.0, against an ideal of 4. Going from d = 64 to d = 128 at N = 4096 takes it from 2166 ms to 7827 ms. That is a factor of 3.6, even though the FLOP count only doubles. The extra cost is in step 3, which reads V down a column. At d = 128 the stride is 512 B and V is 2 MB, too big for the 1.25 MB L2 of a P-core, so most of these strided loads now come from L3. GFLOP/s drops from 1.98 to 1.10 because of this.

Roughly in order, this is where the time goes.

1. Step 1 isn't vectorised. Its inner loop over k is a dot product, a chain of dependent additions into `acc`. Without `-ffast-math` the compiler can't reassociate float additions, so it emits one scalar FMA per iteration (`vfmadd132ss` in the assembly) and each one waits 4 cycles for the one before. A single chain gives 0.5 FLOP/cycle. I measured 2.1 GFLOP/s at about 3 GHz, which is closer to 0.7 FLOP/cycle, because the out-of-order core overlaps the chain for the next j, which is independent. The core can do 32 FLOP/cycle.
2. Step 3 reads V down a column. Its inner loop over j reads `V[j*d + k]`, a stride of d floats (256 B at d = 64), so every access touches a new cache line and uses only 4 of its 64 bytes. At N = 4096, d = 64, V is 1 MB. That is bigger than L1 and close to the L2 size, so most of these loads come from L2 or L3.
3. The score matrix S doesn't fit in cache. It is N x N floats, 64 MB at N = 4096. It gets written once and read four times (max, exp, normalise, step 3), so about 0.5 GB goes through DRAM just for S.
4. `expf` is a libm call. It costs about 3 ns and runs N^2 = 16.8 M times at N = 4096.

Q3. Single-threaded optimisations

variant	N	d	time (ms)	GFLOP/s	% of 1-core peak	speedup vs naive	AI (FLOP/B)	make check
loops	1024	64	16.50	16.27	11.6%	7.7x	7.6	PASS
loops	2048	64	66.36	16.18	11.5%	8.1x	7.8	PASS
loops	4096	64	322.35	13.32	9.5%	6.7x	7.9	PASS
loops	4096	128	592.54	14.50	10.3%	13.2x	15.6	PASS
tiled	1024	64	7.88	34.07	24.2%	16.0x	7.4	PASS
tiled	2048	64	31.39	34.21	24.3%	17.0x	7.7	PASS
tiled	4096	64	154.19	27.85	19.8%	14.0x	7.8	PASS
tiled	4096	128	213.52	40.23	28.6%	36.7x	15.4	PASS
fused	1024	64	16.38	16.39	11.6%	7.7x	170.7	PASS
fused	2048	64	61.13	17.56	12.5%	8.8x	341.3	PASS
fused	4096	64	306.12	14.03	10.0%	7.1x	682.7	PASS
fused	4096	128	568.89	15.10	10.7%	13.8x	682.7	PASS
best	1024	64	4.60	58.38	41.5%	27.4x	128.0	PASS
best	2048	64	16.72	64.20	45.6%	32.0x	256.0	PASS
best	4096	64	63.81	67.31	47.8%	33.9x	512.0	PASS
best	4096	128	121.89	70.47	50.0%	64.2x	512.0	PASS

The d = 128 speedups are bigger than the d = 64 ones mostly because naive itself is 3.6x slower there (Q2).

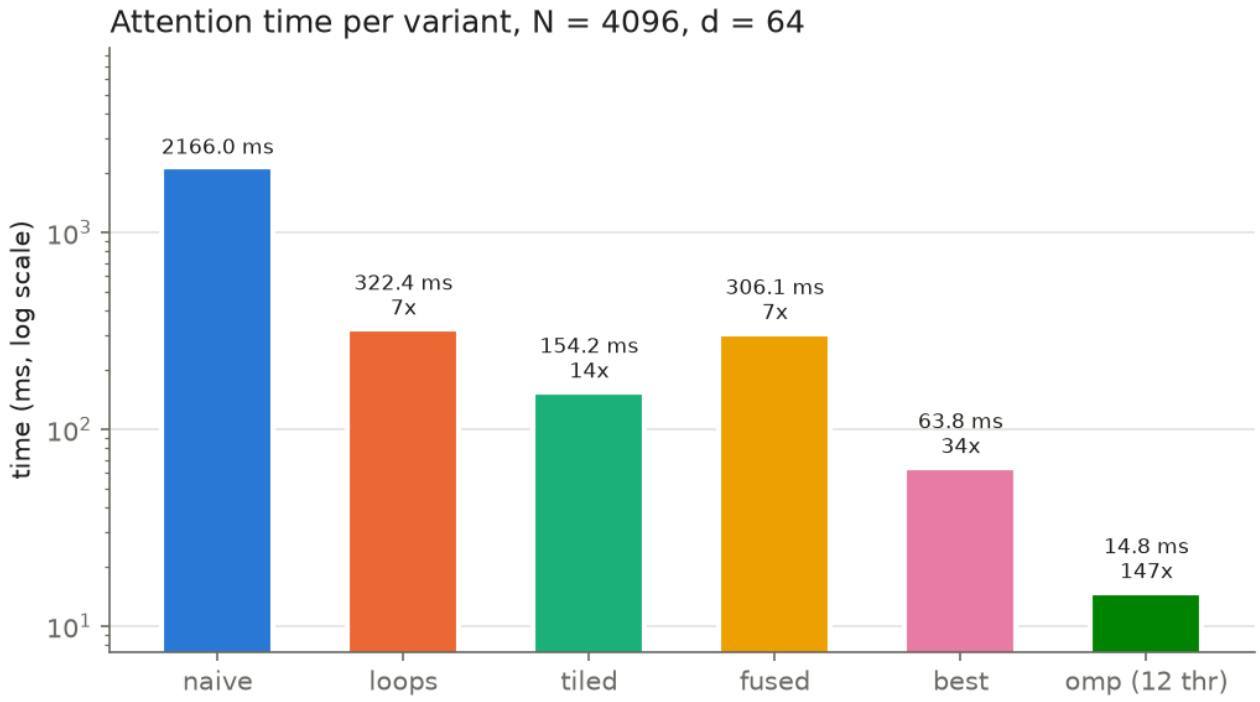


Figure 1: Time of every variant at N = 4096, d = 64 (log scale).

(a) attn_loops.c: loop reordering

I wanted every innermost loop to be unit-stride with independent iterations, so the compiler could vectorise it without help.

Step 1 now runs in i-k-j order over a transposed copy Kt = K^T: `S[i][j] += Q[i][k] * Kt[k][j]`. The inner loop is an AXPY over j. Each iteration writes a different `S[i][j]`, so there is no dependence chain, and the compiler emits 8-wide AVX2 FMAs. I checked the assembly and it uses `vfmadd213ps` on ymm registers. The transpose costs O(Nd), which is nothing next to O(N^2 d). Step 3 becomes i-j-k, `O[i][k] += S[i][j] * V[j][k]`, so V is read along rows.

Loops is 6.7x to 13.2x faster than naive and reaches 9.5% to 11.6% of peak. Each AXPY iteration loads one value of Kt, loads `S[i][j]` and stores it back, three memory operations for one FMA. The core can do two loads and one store per cycle, so the product loops are capped at one vector FMA per cycle, about 48 GFLOP/s at 3 GHz, half of what the two FMA units could do. Loops gets only 13 to 16 GFLOP/s overall, well under that cap, so most of its time is outside the product loops: 16.8 M scalar `expf` calls in the softmax and the DRAM passes over the 64 MB S. Inside the product loops the S row (16 KB at N = 4096) stays in L1, but Kt comes from L2 for every row of Q.

(b) attn_tiled.c: cache and register tiling

Both products are computed in 4 x 16 output tiles. A tile's 64 accumulators stay in eight ymm registers for the whole reduction. Each step loads 2 vectors, broadcasts 4 scalars and does 8 FMAs, so it is 6 loads per 8 FMAs with no stores, compared with 3 memory operations per FMA before. I packed K and V so the 16 values the kernel needs at each step are contiguous (Kp[N/16][d][16], Vp[d/16][N][16]).

For cache tiling, step 1 walks the key columns in blocks of 256. Their packed panels (64 KB at d = 64)

stay in L2 while every row of Q goes over them. Step 3 works on blocks of 256 rows of V, and the 256 x 16 slice of Vp (16 KB) stays in L1 while all the row tiles use it.

Tiled is 14.0x to 36.7x faster than naive and 2.1x to 2.8x faster than loops. The matrix products are much faster than in loops, and most of the remaining time is in the parts tiling didn't touch. S is still stored, so the softmax makes four full passes over 64 MB in DRAM and calls scalar `expf` 16.8 M times. Tiled gets faster with d (27.85 GFLOP/s at d = 64, 40.23 at d = 128) because the product work doubles while the softmax cost stays the same. Its time only grows 1.4x when d doubles.

(c) attn_fused.c: fusion with online softmax

Each row of Q is handled on its own. It walks K and V in blocks of 256 and keeps a running max m, a running sum l and an output vector o:

m' = max(m, max_j s_j), a = exp(m - m'), l <- a l + sum_j exp(s_j - m'), o <- a o + sum_j exp(s_j - m') V_j

At the end O_i = o / l. The factor a rescales whatever was accumulated against the old max, so the result is exactly softmax(s) V up to rounding. The `make check` error is at most 6.2e-6. The only temporaries are a 256-float score buffer and the d-float o. S never exists, and DRAM traffic falls from about 540 MB to about 6 MB at N = 4096.

Fused is 7.1x to 13.8x faster than naive. That is about the same as loops (1% to 9% faster) and half the speed of tiled or less. Removing S fixed something that wasn't the bottleneck any more. Fused has no register tiling, since its inner loops are the same AXPYs as in attn_loops.c, and it still calls scalar `expf` N^2 times. It also reloads all of K and V for every row of Q. At d = 64, K and V together are 2 MB, more than L2, so each row streams 2 MB from L3. That comes to about 8 GB of L3 traffic, an intensity of 0.5 FLOP/B at that level.

(d) attn_best.c

attn_best.c uses the structure of fused with the kernels of tiled, written with AVX2 intrinsics.

1. Rows of Q go in blocks of 32 and K/V in blocks of 64. For each pair I compute the 32 x 64 score tile into L1, then update the softmax state of the 32 rows and the 32 x d accumulator. Each K/V block is loaded once and used by 32 rows, so K and V are read N/32 times instead of N times. That cuts L3 traffic by 32x compared with fused. At d = 64 the working set (K block 16 KB, V block 16 KB, score tile 8 KB, accumulator 8 KB) fits in L1.
2. Both products use 4 x 16 register-tiled micro-kernels with 8 accumulators. Each step does 2 loads, 4 broadcasts and 8 FMAs. The PV kernel also folds the rescale by a into the accumulator load.
3. Q is multiplied once by log2(e)/sqrt(d). The scores then come out in base 2, and the softmax needs 2^x with no extra multiply per score.
4. A vectorised exp2 replaces `expf`. It computes 2^x = 2^n 2^f with n = round(x) and |f| <= 0.5. 2^f comes from a degree-6 polynomial with a relative error of about 1e-7, and 2^n is built by writing n + 127 into the exponent bits. It handles 8 values in about 10 instructions instead of one libm call each. The arguments are always <= 0 here (score minus the row max), and I clamp them at -126 to avoid denormals.

Best is 27.4x to 64.2x faster than naive and reaches 41.5% to 50.0% of single-core peak. At N = 4096, d = 64 it is 2.4x faster than tiled. Against the 96 GFLOP/s peak at the clock the machine actually ran at (Q1), it is at 61% to 73%.

Part of the gap to peak is the softmax and rescaling. They aren't FMA work but still take cycles, about

15% of the instructions in the inner loop. Each micro-kernel step also needs 2 loads plus 4 broadcasts for 8 FMAs, which keeps the load ports busy. And every key block ends with horizontal max and sum reductions.

Q4. OpenMP

attn_omp.c parallelises the loop over the 32-row blocks of attn_best.c. Each block has its own score tile, accumulator, m and l, so the blocks are independent. Threads share only read-only data (Qs, Kp, V), and each thread allocates its own scratch. The packing of Q and K is split across threads too. I used `schedule(dynamic, 1)`, because on a hybrid chip a P-core finishes a block faster than an E-core and a static split would make the run wait for the slowest thread.

The sweep ran with `OMP_PROC_BIND=false`. Speedup and efficiency are relative to attn_omp.c on 1 thread.

N	d	threads	time ms	GFLOP/s	% peak (all)	speedup vs 1 thread	efficiency	speedup vs naive
1024	64	1	4.03	66.6	12.7%	1.00x	100%	31x
1024	64	2	1.97	136.0	26.0%	2.04x	102%	64x
1024	64	4	1.07	251.3	48.1%	3.78x	94%	118x
1024	64	6	0.93	288.0	55.1%	4.33x	72%	135x
1024	64	8	0.96	280.5	53.7%	4.21x	53%	132x
1024	64	10	0.94	284.8	54.5%	4.28x	43%	134x
1024	64	12	0.85	315.2	60.4%	4.73x	39%	148x
2048	64	1	16.34	65.7	12.6%	1.00x	100%	33x
2048	64	2	7.96	134.9	25.8%	2.05x	103%	67x
2048	64	4	5.33	201.6	38.6%	3.07x	77%	100x
2048	64	6	4.15	259.0	49.6%	3.94x	66%	129x
2048	64	8	3.72	288.7	55.3%	4.39x	55%	144x
2048	64	10	3.96	271.2	51.9%	4.13x	41%	135x
2048	64	12	3.25	330.0	63.2%	5.02x	42%	164x
4096	64	1	65.59	65.5	12.5%	1.00x	100%	33x
4096	64	2	32.12	133.7	25.6%	2.04x	102%	67x
4096	64	4	20.97	204.8	39.2%	3.13x	78%	103x
4096	64	6	20.85	206.0	39.4%	3.15x	52%	104x
4096	64	8	17.44	246.3	47.2%	3.76x	47%	124x
4096	64	10	16.25	264.3	50.6%	4.04x	40%	133x
4096	64	12	14.77	290.7	55.7%	4.44x	37%	147x
4096	128	1	152.01	56.5	10.8%	1.00x	100%	51x
4096	128	2	61.05	140.7	26.9%	2.49x	124%	128x
4096	128	4	39.28	218.7	41.9%	3.87x	97%	199x
4096	128	6	37.04	231.9	44.4%	4.10x	68%	211x
4096	128	8	34.09	252.0	48.2%	4.46x	56%	230x
4096	128	10	30.93	277.7	53.2%	4.91x	49%	253x
4096	128	12	28.33	303.2	58.1%	5.37x	45%	276x

Scaling is ideal up to 2 threads (2.04x to 2.05x at d = 64). At 4 threads it is still good on the smallest input (3.78x at N = 1024) but drops to about 3.1x on the larger d = 64 inputs. After that it flattens.

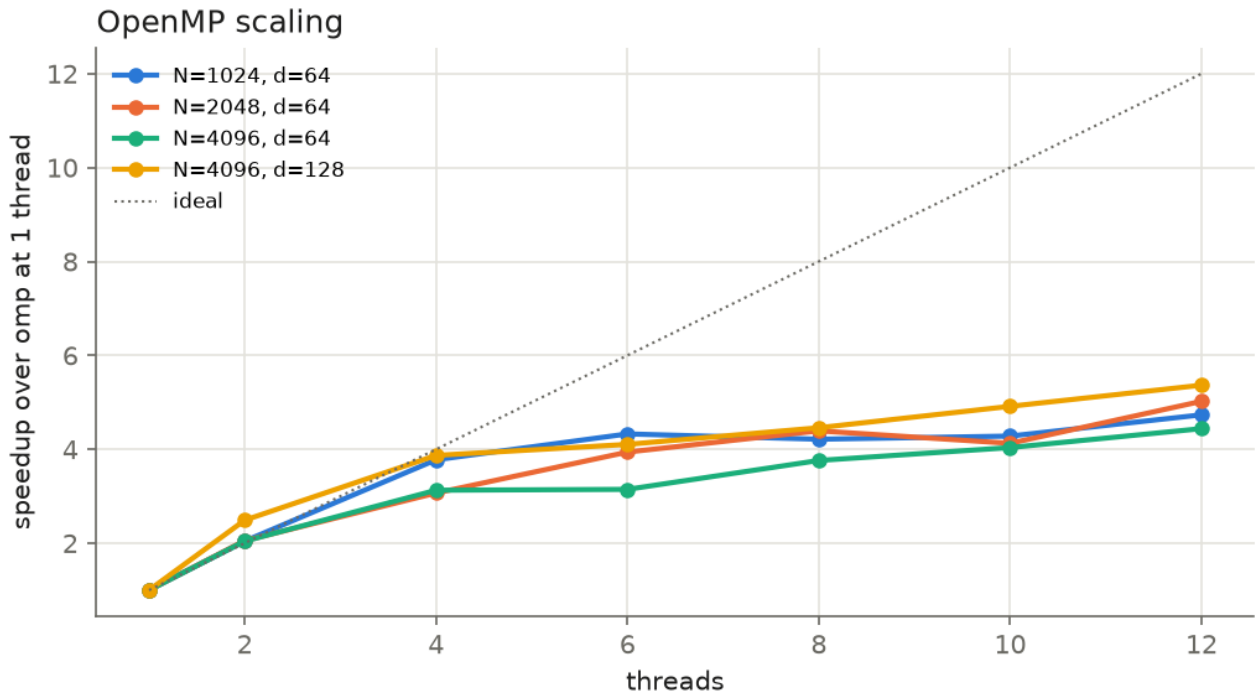


Figure 2: Speedup of attn_omp.c over its 1-thread time.

At N = 4096, d = 64, going from 4 to 6 threads gains almost nothing (3.13x to 3.15x), and 12 threads reach only 4.4x to 5.0x on the d = 64 inputs.

The 12 logical CPUs aren't 12 equal cores. There are 4 P-cores with 2 hardware threads each and 4 E-cores with 1 each, and in the Q1 clock test logical CPUs 6 to 9 were the E-cores. Once the 4 P-cores are busy, an extra thread ends up either on the second hyperthread of a busy P-core or on an E-core. Two hyperthreads share their core's two FMA units. attn_best.c already keeps those units about 70% busy (Q1), so the second thread can only fill the gaps. An E-core does 16 FLOP/cycle instead of 32 and runs at 2.54 GHz instead of 2.81 GHz, so it takes about 2.2x longer per block than a P-core. The whole chip is worth 522.2 / 96 = 5.4 single P-cores, not 12. On the d = 64 inputs the measured 4.4x to 5.0x at 12 threads is 82% to 93% of that ceiling. N = 4096, d = 128 reaches 5.37x only because its 1-thread baseline is slow, as explained in the measurement notes below. At 12 threads omp runs at 55.7% to 63.2% of the all-core peak, compared with about 70% of the measured single-core peak on 1 thread, so a little efficiency is lost per unit of hardware. Under WSL2 the Windows scheduler decides which physical core each logical CPU runs on, and that adds noise. The dip at 10 threads for N = 2048 is within the run-to-run spread.

Dynamic scheduling helps because blocks run at different speeds on different cores. A static split would give every thread the same number of blocks, and the run would wait for the threads on E-cores, which take about 2.2x longer per block. With `schedule(dynamic, 1)`, a thread takes a new block as soon as it finishes one, so the P-cores just end up doing more blocks. The cost is one atomic counter update per block of 32 x N x d x 4 FLOPs.

Memory bandwidth isn't the limit. At N = 4096, d = 64 on 12 threads, DRAM traffic is about 8.4 MB in 14.8 ms. That is 0.57 GB/s, about 1% of 51.2 GB/s. K and V are re-read from L3 once per 32-row block, 268 MB in total, which is 18 GB/s and well within what the L3 can deliver.

N = 1024 is too small. It has only 32 blocks of 32 rows, and each takes about 0.13 ms on one P-core. With 12 threads that is 2.7 blocks per thread, so most threads sit idle in the last round. Fixed costs like waking the threads and packing Q and K are also a large share of a 0.85 ms run. Speedup stays at 4.2x to 4.3x from 6 to 10 threads.

Two notes on the measurements. At N = 4096, d = 128 the 1-thread omp run (152 ms) is slower than attn_best.c (122 ms), so the speedups for that input are inflated, and that is why its 2-thread efficiency is 124%. I repeated the 1-thread runs and got 135 to 164 ms for omp and 122 to 126 ms for best, so the gap is real and not noise. I didn't find the cause. I also tried pinning the run with `taskset -c 0`, but under WSL2 that pins to a virtual CPU rather than a physical P-core, and the times weren't any more stable, so I kept the original sweep.

Q5. Roofline

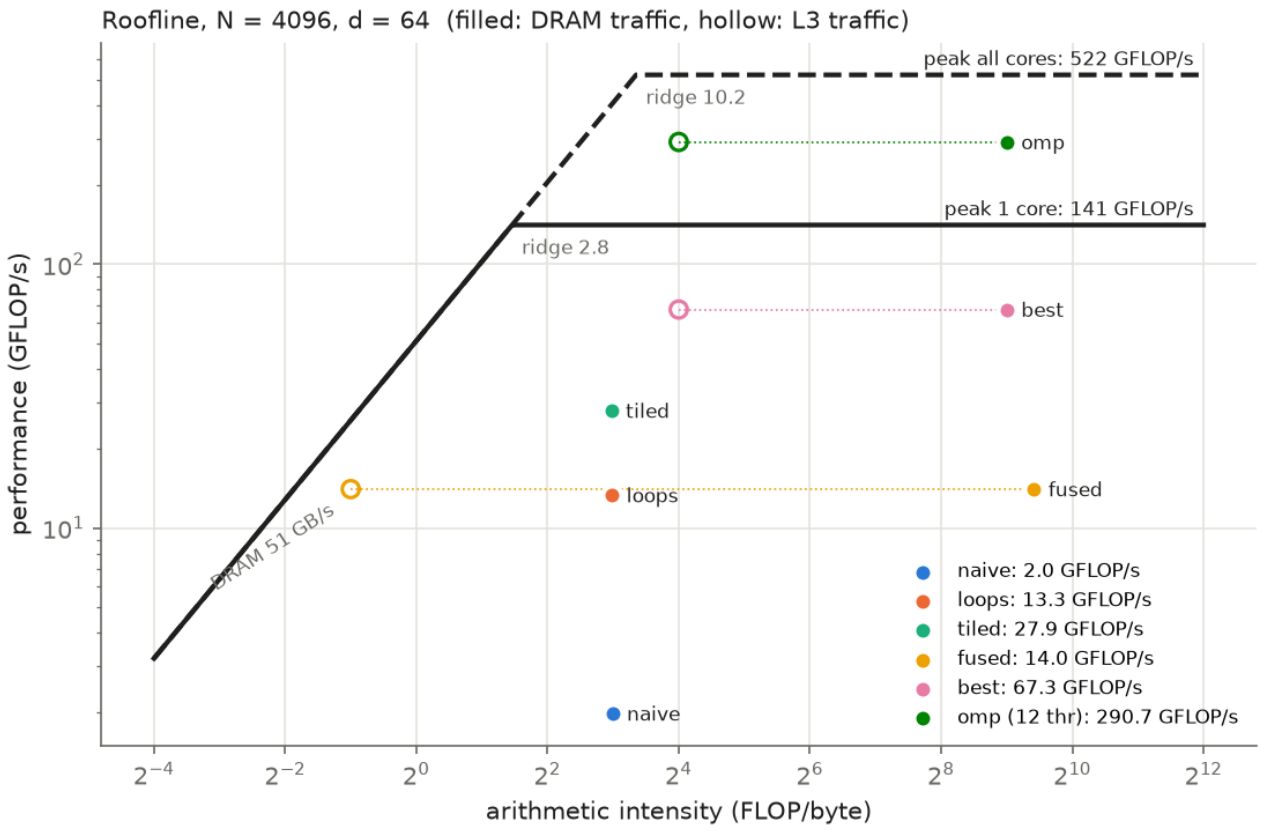


Figure 3: Roofline at N = 4096, d = 64. Filled markers use DRAM traffic, hollow markers L3 traffic.

The roofline uses the Q1 numbers: a 51.2 GB/s slope and flat roofs at 140.8 GFLOP/s (one core) and 522.2 GFLOP/s (all cores). Each variant is plotted at its measured GFLOP/s for N = 4096, d = 64.

Every variant does the same 4N^2 d = 4.29 GFLOP. I computed intensity as that divided by an estimate of the bytes moved between DRAM and the chip. For every variant this includes reading Q, K and V and writing O (4 MB), plus any packed copies.

- naive, loops and tiled store S. It is 64 MB, far bigger than the 12 MB L3, so every pass over it goes to DRAM. The passes are the write in step 1 (counted twice, since a write first reads the line

in), the max pass (1), the exp pass in place (2), the normalise pass in place (2) and the read in step 3 (1). That is 8 x 64 MB, about 540 MB in total, which gives an AI of about 7.9 FLOP/B.
- fused, best and omp never store S. K and V together are 2 MB, which fits in L3, so DRAM sees them about once. Total traffic is 6 to 8 MB and AI is 500 to 680 FLOP/B.

At the DRAM level the variants land on both sides of the all-core ridge. naive, loops and tiled, at about 7.9 FLOP/B, are right of the one-core ridge (2.75) and left of the all-core ridge (10.2). On one core this model says they aren't memory-bound, yet they are far below the 140.8 GFLOP/s roof, so what limits them is inside the core. If they ran on all cores without removing S, DRAM would cap them at 51.2 x 7.9 = about 400 GFLOP/s, below the all-core roof. fused, best and omp, at 500 to 680 FLOP/B, are right of both ridges.

The DRAM model leaves out cache traffic, so the plot also shows the L3-level intensity of the fused variants as hollow markers. Fused reloads K and V from L3 for every row of Q (8N^2 d bytes, AI 0.5). Best and omp reload them once per 32 rows (AI 16), which is still right of both ridges.

- naive (1.98 GFLOP/s) is held back inside the core by scalar dependent FMAs and strided loads. It is far from both roofs.
- loops (13.32) is vectorised, but the load/store traffic in its AXPY loops limits its product loops to about one FMA per cycle, but most of its time goes to scalar `expf` and the DRAM passes over S.
- tiled (27.85) has fast products, but most of its time now goes to the DRAM passes over S and to scalar `expf`. Its DRAM point is right of the one-core ridge and left of the all-core ridge, so a parallel version that kept S would hit the bandwidth slope before the compute roof.
- fused (14.03) has removed DRAM traffic, but at the L3 level its intensity of 0.5 is below any reasonable L3 ridge, and it still uses scalar `expf`. It is bound by cache bandwidth and latency.
- best (67.31) and omp (290.73) are compute-bound at every level. They sit 47.8% and 55.7% of the way up their roofs.